

Report shell project

Operating Systems Concepts

Names & student numbers:

- Bartu Saglamer, 1197922
- Christiaan Smits, 1193879

1. Implementation

The shell reads one line with `getline()`, parses it into an `Expression`, and executes that expression. The parser first splits the line at `|` and stores every command as a `Command` containing its argument strings. It then removes a trailing `&`, `< file`, or `> file` where these operators are allowed, and stores the corresponding information in the `Expression`.

For a single command, the shell handles `cd` and `exit` in the shell process. Other commands are executed by a child created with `fork()`. The child applies input and output redirection with `open()` and `dup2()`, and then calls `execvp()`. Unless the command is a background command, the parent waits for the child with `waitpid()` before displaying the next prompt. For an empty input line, the shell does nothing and continues with the next prompt.

For a pipeline, the executor loops over `expression.commands`. Before every command except the last, it calls `pipe()`. It then calls `fork()` for the command. In the child, the previous pipe read end is connected to standard input with `dup2()`, and the current pipe write end is connected to standard output with `dup2()`. The first command can instead receive input from the file named by `<`; the last command can instead send standard output to the file named by `>`. The child closes unused descriptors and calls `execvp()`. The parent closes its unused descriptors, saves every child PID, and starts all commands concurrently. For a foreground pipeline it then calls `waitpid()` for every child. For a background pipeline it returns immediately to the shell loop.

Completed background processes are reaped in the shell loop using `waitpid()` with `WNOHANG`. This prevents zombie processes without blocking the shell while background commands are still running.

The system calls and related functions used are:

- `getcwd()`: obtains the current directory for the prompt.
- `chdir()`: changes the shell's directory for `cd`.
- `fork()`: creates a child for each external command.
- `pipe()`: connects the output of one pipeline command to the input of the next.
- `dup2()`: replaces standard input or output with a pipe or file descriptor.
- `open()`: opens input files and creates or truncates output files.
- `close()`: releases unused file descriptors so pipes can reach EOF.
- `execvp()`: replaces a child with the requested external program.
- `waitpid()`: waits for foreground children and reaps them.
- `exit()` and `_exit()`: terminate the shell or a child after an execution error.

The built-in commands `cd` and `exit` are handled directly by the shell process only when they are used as standalone commands. This is necessary because `cd` must change the working directory of the shell itself, while `exit` must terminate the shell itself. Commands inside a pipeline are executed in separate child processes.

2. Tests

The shell was tested from `test-dir`, because the test input files and expected relative paths are located there.

command	expected result	actual result
<code>pwd</code>	print the current working directory	correct directory printed
<code>echo a b c d</code>	print all four arguments	correct output printed
<code>ls -l</code>	list the test files, one per line	correct listing printed
<code>date</code>	print the current date and time	command completed and printed output
<code>date \ tail -c 5</code>	print only the final five characters	correct final characters printed
<code>cat < 1</code>	print all four lines from file <code>1</code>	correct file contents printed
<code>ls -l \ head -n 2</code>	print only the first two entries	correct two entries printed
<code>cat < 1 > ../foobar</code>	write file <code>1</code> to <code>../foobar</code> and print no regular output	file was created with the correct contents
<code>cat < 1 \ head -n 3 \ tail -n 1 > ../foobar</code>	write only line 3 to <code>../foobar</code>	file contained <code>line 3</code>
<code>sleep 1 &</code>	return to the prompt without waiting	prompt returned immediately
<code>does-not-exist</code>	display an error and continue	error was displayed and the shell continued
<code>cd ..</code> followed by <code>pwd</code>	change the shell's working directory	directory changed correctly
<code>cd / \ ls</code> followed by <code>pwd</code>	execute the pipeline without changing the shell's directory	pipeline executed and the shell remained in the original directory

The provided GoogleTest target could not be rebuilt on the current system because its vendored CMake configuration requires an obsolete CMake compatibility version. The shell executable itself compiled successfully, and the functional commands above were run manually.

The tests do not prove that the implementation is free of faults. An error is a human mistake, such as forgetting to close an unused pipe descriptor. This can introduce a fault in the source code, such as keeping a pipe write end open. When this fault is executed, a command may wait forever for EOF. This observable

incorrect behaviour is a failure. More tests can increase confidence, but no finite test set can cover all inputs, scheduling orders, resource limits, or operating-system failures.

3. Infinite buffer problem

An operating system cannot provide an infinite communication buffer because memory and kernel resources are finite. A pipe therefore has a finite kernel buffer. If a producer writes faster than its consumer reads, the producer blocks when the pipe is full. If the pipe is empty, the consumer blocks until data arrives. This flow control prevents unlimited memory use.

The shell does not need an infinite buffer. It creates all processes in a pipeline before waiting and connects them directly with pipes. Data is passed as a stream while the producer and consumer run concurrently; the shell does not collect the complete output in its own memory. When a command finishes, its pipe write end is closed. After all write ends are closed, the next command receives EOF and can terminate, assuming the commands themselves terminate.

The placement of `waitpid()` matters. Waiting for the first command before starting the next one could deadlock when the first command produces more data than the pipe can hold. This implementation starts every pipeline command first and waits only after the pipeline has been connected, so the finite pipe buffers are sufficient.